

Code Quality Report — Tick & Taka (round 4)

Bun monorepo: Hono API, Expo Android app, shared package

Project	Tick & Taka (tick-taka)
Date	2026-10-07
Commit	801dd5f (branch fix/audit-2026-10-06)
Previous audits	Round 1 at 3b5438b : 70/100. Round 2 at df29d67 : 72/100. Round 3 at 7e2bfefb : 83/100, 10 open findings. 36 commits since round 3.
Auditor role	Staff Engineer (architecture, maintainability, security, performance)
Companion reports	QA_Report.pdf and UX_Report.pdf, written in parallel by other auditors
Mode	Read-only. No source, config, dependency or git state was changed. Scan and probe scripts ran from docs/audit/2026-10-07-r4/_cq_tmp/ and were deleted afterwards.

1. Executive summary

89

/ 100

R1 70 · R2 72 · R3

83

The 36 commits fixed what round 3 asked for, and the tests now feed in the inputs production actually produces. A reply with no `x-server-time` header can no longer reset the edit clock to 1970. The phone ignores it, the saved offset is checked on restore, and the server applies an edit stamped before 2024 instead of quietly dropping it. Stuck writes now take the writes that depend on them along. Save failures show in the sync bar. The widget keeps its own copy of a write until the outbox has saved it, and the screen cache survives a Keystore failure without ever being written in plain text. Of the 10 open round-3 findings, 7 are fixed, 3 are partly fixed, and none were ignored or regressed.

No High was found. The one Medium comes from the stuck-write fix itself. A write counts as dependent on a stuck one if it *names* the same record, even when it doesn't need it. So a category rename the server keeps refusing holds back every later expense in that category, and tapping Discard throws those expenses away along with the rename (reproduced). The six Lows are small. The CI audit passes when `bun audit` can't reach the registry (reproduced). The API's own `http_error` code looks to the phone like an error page from a proxy. Two exported hooks are both named `useToday`. And three round-3 items are only partly done.

Severity (open findings)	Count	Round-3 status	Count
Critical	0	Fixed	7
High	0	Partially fixed	3
Medium	1	Not fixed	0
Low	7	Regressed	0
Total	8 (5 new, 3 carried)		

Top 5 issues to fix first

ID	Severity	Issue
CQ-043	Medium	A stuck edit of a category or account holds back every later expense that names it. Retry can't get them through, and Discard drops them with the edit. The sync bar labels the group with the edit's error text (reproduced). Effort M.
CQ-044	Low	The CI audit script exits 0 when <code>bun audit</code> fails to run, and suggests removing every accepted advisory (reproduced). Effort S.
CQ-045	Low	The API sends <code>http_error</code> for its own HTTPExceptions (such as malformed JSON). The phone reads that code as "a proxy answered", so the write is retried forever and the user is told they're offline (reproduced). Effort S.
CQ-037	Low	Carried: if SecureStore fails to save a refreshed token, the error is swallowed. The token is kept only in memory and the next cold start asks the user to sign in.
CQ-046	Low	"Today" is worked out three ways on the phone, through two exported hooks both named <code>useToday</code> that return different types.

2. Scorecard

Area	R1	R2	R3	R4	Justification
Architecture	8	8	8	8	The splits follow real seams. quick-add is now three siblings, imported by name with no barrel. Widget layout is a pure module, and the cache wrapper is a small Persister decorator. The phone now has three "today" helpers (CQ-046), and the lazy schema cycle is still the only import cycle.
Readability & maintainability	7	7	8	8	No file is over 800 lines, and there are 0 TODOs and 0 dead value exports. The month stepper, icon button, toggle row and date field are shared components with named variants. But <code>outbox-queue.ts</code> grew 59% to 534 lines and does three jobs. ToggleRow isn't used in three sheets, and one orphaned doc line is still there (CQ-047, CQ-041).
Type safety	8	7	8	9	Error bodies are now parsed with the shared <code>errorBodySchema</code> , and the clock header with <code>serverTimeOf</code> . There are 0 <code>!</code> , 1 justified <code>any</code> , 0 <code>@ts-ignore</code> and 143 <code>as</code> casts (144 in R3). The assistant's settings allow-list is typed <code>Set<SettingsKey></code> , so a typo won't compile.
Error handling	5	6	8	8	Save failures are reported once and shown until a save works. Too-large replies (413/414/431) are final. Two gaps are left: the API and the phone both use the <code>http_error</code> code (CQ-045), and a failed token save is swallowed (CQ-037).
Security	6	6	8	9	The screen cache is never written in plain text. The assistant can no longer change app lock, AI, currency, accounts or time zone. The plain-text export is deleted after sharing and on sign-out. One gap: the advisory check fails open in CI (CQ-044).
Performance	6	8	9	9	A refreshed session now writes once and then only reads (tested by counting UPDATES). Sync pulls ask for counts, not rows. No new hot paths were found.
Data layer & sync integrity	7	8	7	8	Both round-3 sync bugs are fixed with real-input tests. Unpay needs nothing from the pay's reply and is safe to repeat. Budget drafts write one line. Against that, stuck-group parking now pulls in writes that don't depend on the stuck one (CQ-043).
Tooling & DX	6	8	8	8	Biome, typecheck and 774 tests are green. CI audits against an allow-list, and 36 of 36 commits are Conventional, with no attribution lines and 35 Refs: trailers. The audit script itself fails open (CQ-044), and nothing typechecks <code>scripts/</code> .
Testing	6	7	8	8	The idealised inputs are gone: a <code>ky Response</code> without the header, the real TanStack persister on a cipher that can be locked, and a stuck create followed by its PATCH and DELETE. <code>ai/ask.ts</code> is still at 62.7% lines and <code>describeDraft</code> has no test (CQ-042).
Dependencies	7	7	8	8	uuid is pinned through overrides. The 5 tooling advisories are accepted with a reason each in <code>scripts/audit-allowlist.json</code> . Bun is pinned and installs use a frozen lockfile.

The overall score is a weighted judgement, not an average. Security, error handling and data integrity count double, because the app holds personal finance records. Round 3 put the codebase "close to 90" once CQ-036 and CQ-038 were fixed. Both are fixed, along with five Lows. The CQ-038 fix brought in a Medium of the same kind, on the same path, so the score stops just under 90. With CQ-043 fixed, the codebase is in the low 90s.

3. Scope & method

What was reviewed

- Every open round-3 finding, re-checked line by line at 801dd5f (section 4).
- Everything in `git diff 7e2bfeb..HEAD` (138 files, +4,498/−1,412). That covers `server-clock.ts`, `http.ts` and `event-stream.ts`; the pre-2024 branch in `crud.ts`; and in the outbox, the stuck groups, cancel, the side key, `durable()` and save-failure reporting. Also: `recurring/service.ts` `unpay` and `recurring-pay.ts`; the error envelope and `CACHE_BUSTER`; `cache-persister.ts` and `secureCacheStorage`; export clean-up; the per-line budget route; the assistant settings allow-list and Undo; `sessions.touch`; the quick-add split; `scripts/audit.ts` and CI; the shared UI components (month-stepper, icon-button, date-field, toggle-row, picker-options, chart-axis, calendar-grid); `widget-layout.ts` and the widget hand-over; `use-today.ts`; and all new and changed tests.
- Whole-repo scans: file and function sizes and type-safety counts (TypeScript AST), import cycles (Tarjan SCC over 425 source files, runtime imports only), duplicate 8-line blocks, exports referenced nowhere, `console.*`, swallowed catches, TODOs and committed secrets.
- Yardstick: README.md, PRODUCT.md, DESIGN.md, docs/IMPLEMENTATION_PLAN.md, the spec (`_raw/spec.txt`), the round-3 report, and the owner's rules: an 800-line ceiling, no barrel files, strict TypeScript, Conventional Commits, and a shared component with named variants for markup repeated in 3 or more files.

Commands run

Command	Result
<code>bunx biome check .</code>	555 files, 0 diagnostics, exit 0 (<code>_raw/lint.txt</code>)
<code>bun run typecheck</code>	shared, api, mobile: exit 0
<code>bun run test</code>	shared 247, mobile 291, api 236 pass, 0 fail (774 in all; R3 689)
<code>bun test --coverage per workspace</code>	api 94.50% lines / 88.20% funcs; shared 95.08 / 84.16; mobile 93.05 / 90.97
<code>bun run audit</code>	5 accepted, no new advisories, exit 0 (<code>_raw/audit.txt</code>)
<code>bunx expo export --platform android</code>	Bundles OK: 2,855 modules, 7.7 MB Hermes bytecode
AST scan (TypeScript 6.0.3 compiler API)	143 <code>as</code> casts, 96 <code>as const</code> , 0 non-null <code>!</code> , 1 <code>any</code> , 0 <code>@ts-ignore</code> ; sizes in Appendix C
Import-graph SCC	1 cycle: <code>db/schema/time.ts</code> ↔ <code>db/schema/money.ts</code> (unchanged, lazy)
<code>bun scripts/audit.ts</code> with the registry pointed at 127.0.0.1:9	Reproduced CQ-044: bun audit fails ("ConnectionRefused"), yet the script prints "No new advisories" and exits 0

Command	Result
Probe <code>probe.test.ts</code> (Appendix G)	Reproduced CQ-043 (a stuck PATCH <code>/categories/C</code> holds back two expenses in C) and CQ-045 (an API envelope with code <code>http_error</code> maps to <code>{unreachable: true}</code> and "retry", which never counts as a server fault)

Not verified

Unverified No device run. The emulator image has no Google Play services, and sign-in is Google-only. How often Keystore reads or SecureStore writes fail (CQ-037) and how often a server bug produces a stuck write (CQ-043) could not be measured.

Unverified Export clean-up timing. `use-export-data.ts:26-33` deletes the shared file as soon as `shareAsync` resolves. On Android, that happens in `onActivityResult` (`expo-sharing` `SharingModule.kt:80-85`), when the target activity returns. A target that reads the content URI later, such as a background upload, would then find the file gone. Whether any common target does this needs a device.

Unverified The production server: whether `.env` sets `TRUST_PROXY=true` and `BACKUP_OFFSITE_CMD`, and whether the installed Nginx site matches `deploy/nginx-tick-taka.conf`.

Unverified I did not run `bun outdated` (it needs the network), and I could not check the allow-list's claim that braces and node-forge have no patched release.

4. Previous findings status (round 3 → round 4)

Each row was checked against the code at `801dd5f`. Line numbers refer to `801dd5f`.

ID	Round-3 finding	Status	Evidence
CQ-036	A reply without <code>x-server-time</code> sets the edit clock to 1970 High	Fixed	<code>serverTimeOf</code> returns NaN for a missing or empty header (<code>server-clock.ts:39-42</code>). <code>noteServerTime</code> ignores anything that isn't finite or is before <code>EARLIEST_PLAUSIBLE_MS</code> (2024-01-01, :29-54). Both request paths use it (<code>http.ts:73</code> , <code>event-stream.ts:39</code>). On restore, an offset that puts "now" before 2024 is rejected (:57-72). On the server, a stamp before 2024 counts as no stamp, so the edit applies in arrival order (<code>crud.ts:86-90</code>). Tests use real inputs: <code>Number(new Headers().get(...))</code> and a ky request answered by a header-less 502 (<code>server-clock.test.ts:40-94</code>), plus <code>time.test.ts</code> on the API. One recommendation was skipped, keeping <code>editTime()</code> from ever going backwards. It's optional now that bad offsets are refused.
CQ-038	Parking a stuck write lets dependent writes go first Medium	Fixed	<code>park()</code> moves every later entry that names an id the stuck write touches into its group (<code>outbox-queue.ts:424-443</code>). New writes join the group when they're queued (:260-276, 446-456). Retry and Discard act on the whole group, and Discard has an Undo (:116-172, <code>sync-status.tsx:62-79</code>). The test covers exactly the case asked for: a stuck POST, then its PATCH and DELETE, with an unrelated write that goes through (<code>outbox-queue.test.ts:327-379</code>). The way "dependent" is matched is too broad, though; see the new CQ-043.
CQ-039	Silent outbox save failures; widget hand-over drops its durable copy Low	Fixed	Saves are chained through <code>queueSave</code> , which reports the first failure, shows it until a save works and exposes <code>notSaved</code> (<code>outbox-queue.ts:514-529</code>). <code>outbox.ts:45-50</code> logs it and notifies the user, and the sync bar names it (<code>sync-status.tsx:112-113</code>). Writes queued before the saved queue is read go to a side key (<code>tt.outbox.early</code> , :493-512) and are merged in on load. The widget trims its own lists only after <code>durable()</code> is true (<code>widget-pending.ts:32-47</code>). Tests: <code>outbox-durability.test.ts</code> , <code>widget-handover.test.ts</code> .
CQ-037	Key failure at start discards the cache; plain-text fallback Low	Partially fixed	The cache half is done well. <code>keepCacheOnKeyFailure</code> turns <code>KeyUnavailableError</code> into "restore nothing", won't save over the kept cache, and merges it in once the key reads again (<code>cache-persister.ts:14-58</code>). The cache storage has <code>plainFallback: false</code> (<code>secure-storage.ts:67-76</code>). It's tested against the real TanStack persister (<code>cache-persister.test.ts</code>). The token half is untouched: <code>refreshIfStale</code> still swallows a failed <code>SecureStore</code> write (<code>session-manager.ts:66-76</code> , 108-127). Carried, narrowed.

ID	Round-3 finding	Status	Evidence
CQ-042	Tests use inputs the real code never passes Low	Partially fixed	Three of the four Given/When/Then cases now exist, all with real inputs: the header-less reply, the stuck create with its dependants, and the key failure at cache restore. <code>pullChanges (sync-pull.test.ts)</code> and <code>turn recovery (turn-recovery.test.ts)</code> are covered too. The fourth case, the "ask my data" tools, is not: <code>ai/ask.ts</code> is still at 62.68% lines, with 114-187 uncovered. Carried, narrowed.
CQ-041	Small leftovers Low	Partially fixed	5 of the 6 actionable items are done. <code>requireAuth</code> has its own doc again (<code>auth.ts:17-22</code>). <code>expireSession</code> is gone. <code>offsite</code> uses <code>errorFields (offsite.ts:37)</code> . <code>touch()</code> reads before it writes (<code>sessions.ts:110-119</code> , tested by counting UPDATES). The README now says which assistant changes have Undo (README.md:124-127), and settings gained one. Still open: the stranded doc line at <code>outbox-policy.ts:86</code> . The lazy schema cycle is unchanged and harmless. Carried, narrowed.
CQ-040	Month stepper in 3 files; no IconButton Low	Fixed	<code>MonthStepper (components/ui/month-stepper.tsx)</code> is used by all three screens. <code>IconButton</code> has the named shapes <code>plain</code> and <code>round</code> and is used at 11 sites. <code>ToggleRow</code> , <code>DateField (named kind/span)</code> , <code>useChartAxis</code> and <code>CALENDAR_GRID</code> follow the same rule. The duplicate scan finds no block shared by 3 or more files. A remainder is in CQ-047.
CQ-020	<code>quick-add.ts</code> at 649 lines and growing Low	Fixed	Split along the seams named in round 3: <code>quick-add-money.ts (251)</code> , <code>quick-add-task.ts (160)</code> , <code>quick-add-guess.ts (78)</code> , and <code>quick-add.ts (186)</code> for the entry point and preview. The siblings are imported by name, with no barrel, and they import back only types.
CQ-022	Client JSON casts; cache buster never bumped Low	Fixed	Error bodies are parsed with the shared <code>errorBodySchema (api.ts:34-38)</code> , and <code>event-stream.ts</code> uses the same reader. <code>CACHE_BUSTER</code> is a named constant at "3" with a written bump rule and a changelog (<code>query-client.ts:100-110</code>). It was bumped twice this round, the second time in 6010dc9, when the phone started sending Today's <code>dueAt</code> . The remaining <code>as</code> in <code>unwrap</code> is the typed-RPC contract. Bumping the buster is a matter of discipline; nothing enforces it.
CQ-012	CI audit job always red Low	Fixed	<code>scripts/audit.ts</code> reads <code>bun audit --json</code> and fails only on a GHSA id that isn't in <code>scripts/audit-allowlist.json</code> . CI runs it without <code>continue-on-error (check.yml:19-28)</code> , and the README points to it. The script fails open when the audit itself can't run; see the new CQ-044.

5. Findings

Grouped by category and sorted by severity. New findings are numbered from CQ-043. Carried findings keep their earlier IDs. Effort: S < 1 h, M < 1 day, L > 1 day.

5.1 Offline sync & error handling

CQ-043 · A stuck edit holds back every later write that names its record, and Discard drops them with it

Medium · Offline sync / data integrity · Effort M · New · Reproduced (probe B)

Where

apps/mobile/src/lib/outbox-policy.ts:103-136 (`writtenIds`, `mentionsAny`),
apps/mobile/src/lib/outbox-queue.ts:138-146, 260-276, 424-456,
apps/mobile/src/components/ui/sync-status.tsx:19-26, 52-79

What's wrong

To fix CQ-038, a stuck write's group is defined by *mention*. `writtenIds` takes every ULID in the stuck write's path, plus `id/transactionId` in its body. `mentionsAny` then matches any later write that carries one of those ids anywhere in its path or body, up to 3 levels deep. That rule is right when the stuck write *creates* the record, because a later edit would get 404. It's wrong when the stuck write only edits a record the server already has: `PATCH /categories/C`, `PATCH /accounts/A`, `PUT /budgets/2026-10/C` (the category id is in the path), `POST /recurring/R/pay`. Every later expense with `categoryId: C` or `accountId: A` is set aside too, though the server would accept it, and so is every one queued after the park (`parkedHeadFor`).

Getting out isn't easy. Retry sends the failing head first, so the group is parked again after another 8 server faults (about two minutes). The only other way out is Discard, which drops the whole group (`discardStuck`), expenses included. The bar describes the group by the head's `label`, and labels are written as rejection messages. The row therefore reads "Couldn't save, and 2 later changes to it", and the snackbar says "Discarded "Couldn't save"". Nothing tells the user that two expenses are in the group.

Evidence

```
// outbox-policy.ts:132-135: any id, anywhere in path or body
export function mentionsAny(request: OutboxRequest, ids: ReadonlySet<string>): boolean {
  if (ids.size === 0) return false;
  const inPath = (request.path.split("?", 1)[0] ?? "").split("/").some((s) => ids.has(s));
  return inPath || bodyMentions(request.body, ids, 0);
}
$ bun test probe.test.ts      (stuck: PATCH /categories/C answered 500 every time)
probe B: {"STUCK_AFTER":8,"sentTransactions":0,
  "stuck":["PATCH /categories/01M4...Q50","POST /transactions","POST /transactions"]}
```

Two expenses in category C, one queued before the park and one after, were never sent. All three writes sit in one group.

Why it matters

Expenses are the core capture flow. While they're held back, safe-to-spend on the server, the widget and any other device are wrong. The one action that unblocks the queue throws them away. The trigger is the same as CQ-038's, a write the API keeps failing with 5xx (a server bug), so it's

rare. When it happens, though, it drops money records, and the UI doesn't tell the user that's what Discard will do.

Recommendation

Make a write depend on the ids the stuck group *creates*, not on every id it mentions. For a stuck write that isn't a create, park only later writes to the same record (its path, or a sub-path of it). Describe the group by kind rather than by its error label, and say what Discard drops.

```
// outbox-policy.ts
export function createdIds(r: OutboxRequest): string[] { // POST that brings a record into
  being
  const id = (r.body as { id?: unknown } | undefined)?.id;
  return r.method === "POST" && typeof id === "string" && RECORD_ID.test(id) ? [id] : [];
}
export function sameRecord(r: OutboxRequest, head: OutboxRequest): boolean {
  const base = head.path.split("?", 1)[0] ?? "";
  return r.path === base || r.path.startsWith(`${base}/`);
}
// park(): depends = mentionsAny(later, createdIds(group)) || sameRecord(later,
  entry.request)
```

Add a test: given a stuck PATCH /categories/C, when POST /transactions {categoryId: C} is queued, then it is sent. Keep the existing test for a stuck create.

Effort

M

CQ-045 · The API's own http_error code is read by the phone as "a proxy answered", so such a write is retried forever

Low · Error handling / API contract · Effort S · New · Reproduced (probe A)

Where

apps/api/src/middleware/error-handler.ts:17-19,
node_modules/hono/dist/validator/validator.js:18 (hono 4.13.13),
apps/mobile/src/lib/api.ts:34-37, apps/mobile/src/lib/outbox-failure.ts:15,
apps/mobile/src/lib/error-copy.ts:62

What's wrong

87daf33 made the phone tag any error body that fails `errorBodySchema` with code `http_error`, meaning "something in front of the API answered". `describeFailure` turns that code into `{unreachable: true}`. But the API already sends `http_error` in its own, valid envelope for every `HTTPException`. Hono's JSON validator throws one, `HTTPException(400, "Malformed JSON in request body")`, on any `validate("json")` route. The phone checks the code string, not whether the body parsed, so it can't tell the two apart. A permanent 400 becomes "retry forever". It never counts as a server fault, so it is never parked, and the error copy says the user is offline.

Evidence

```
// error-handler.ts:17-19
if (error instanceof HTTPException) {
  return c.json(errorBody("http_error", error.message || "Request failed"), error.status);
}
// outbox-failure.ts:15
if (error.code === "http_error" && error.status !== 401) return { unreachable: true };
probe A: {"code":"http_error","info":{"unreachable":true},"kind":"retry","serverFault":false}
```

Why it matters

Today's trigger is rare, because ky serialises the JSON itself. But it's a contract collision. Any future `HTTPException`, such as a body limit or a header check, would silently block its write at the head of the queue forever, and every write behind it would wait in order.

Recommendation

Give the API's code its own name (for example `bad_request`), or map 4xx `HTTPException`s to `validation_error`. On the phone, mark the "not our envelope" case with a field of its own (for example `ApiError.fromProxy = true`) instead of reusing a code string the server also sends. Add a test that a valid envelope with status 400 and any code is classified `reject`.

Effort

S

CQ-037 · If SecureStore fails to save a refreshed token, the next cold start asks the user to sign in (carried, narrowed)

Low · Error handling (client) · Effort S · Partially fixed · Unverified on device (how often)

Where

`apps/mobile/src/lib/session-manager.ts:66-76, 108-127`

What's wrong

The cache half of round 3's finding is fixed (section 4). The token half is unchanged. `save()` sets `issuedAt` and the in-memory token first, then awaits three `SecureStore` writes. `refreshIfStale` calls it inside a `try` whose empty `catch` is meant for "offline". If `setItem(TOKEN_KEY)` throws, the new token exists only in memory, and `issuedAt` says it's fresh, so no retry follows. Meanwhile the old token is retired 60 s after the new one is first used. The next cold start loads the old token, gets `session_expired` and shows the login screen.

Evidence

```
// session-manager.ts:119-124
const session = sessionResponseSchema.safeParse(await response.json().catch(() => null));
if (session.success && tokenStore.get() === token) await save(session.data);
} catch {
  // Offline or unreachable: try again on the next start or foreground.
}
```

Why it matters

The outbox keeps queued writes through a sign-in, so nothing is lost. But an unexpected sign-in with Google is the kind of friction the spec wants to avoid, and nothing is logged to explain it.

Recommendation

Handle a failed `save()` on its own: log it, and set `issuedAt = null` so the next foreground refreshes again (the old token still works until the new one is used). Add a test with a storage fake whose `setItem` rejects.

Effort

S

5.2 Tooling & security

CQ-044 · The CI audit passes when `bun audit` can't run

Low · Tooling / security · Effort S · New · Reproduced

Where

`scripts/audit.ts:19-27, 39-56`, `.github/workflows/check.yml:20-28`

What's wrong

The script's header says it exits 1 "when the audit can't run". But it ignores `audit.exited`, and it reads empty output as `"{}"`. When the registry can't be reached, `bun audit --json` writes nothing to stdout and exits 1. The script then finds 0 advisories, prints a notice to delete each of the 5 accepted ids from the allow-list, reports "No new advisories" and exits 0. A registry outage, a rate limit or a bun change that breaks `--json` all turn the check green.

Evidence

```
// scripts/audit.ts:21-25
const output = await new Response(audit.stdout).text();
await audit.exited; // exit code never read
let report: Record<string, Advisory[]>;
try { report = JSON.parse(output || "{}") ... }
$ BUN_CONFIG_REGISTRY=http://127.0.0.1:9/ bun scripts/audit.ts
ConnectionRefused: audit request failed
::notice::GHSA-vfj7-8cjl-p6xm is accepted ... but no longer reported; remove it. (x5)
No new advisories (0 accepted).
exit=0
```

Why it matters

The job exists to catch a new advisory, and it fails open in exactly the case where it learned nothing. The notices also tell whoever reads the log to empty the allow-list.

Recommendation

Fail when stdout is empty or isn't a JSON object, and when `exited !== 0` while no advisories were parsed (bun exits 1 when advisories exist, so the exit code alone can't decide). Have a run where every accepted id is missing say "audit returned nothing" instead of "remove it". `scripts/` also isn't covered by any `typecheck`; a root `tsconfig` include or a `tsc -p scripts` step would cover it.

Effort

S

5.3 Maintainability

CQ-046 · "Today" is worked out three ways on the phone, and two hooks are both named `useToday`

Low · Maintainability / naming · Effort S · New

Where

`apps/mobile/src/lib/use-today.ts:6-19`, `apps/mobile/src/lib/queries.ts:42-69`, `76-90`, plus 10 inline `toLocaleDate/toLocalMonth(Date.now(), timeZone)` calls: `plan-home.tsx:31`, `logbook-list.tsx:24`, `calendar-view.tsx:39`, `money-calendar.tsx:41`, `time-tracking.tsx:35`, `events-screen.tsx:33`, `insights-screen.tsx:53`, `goal-sheet.tsx:121`, `focus-stats.tsx:28`, `quick-add-requests.ts:31`

What's wrong

1fc2209 correctly moved ten screens to the user's time zone. It did so by adding `useToday()` and `useThisMonth()` in `use-today.ts`, which compute a string at render time. `queries.ts` already exports `useLocalToday()`, which turns over at local midnight and on foreground, and a *different* `useToday(date?)` that returns the Today screen's `UseQueryResult`. Which one an editor auto-imports decides whether the code compiles against a string or a query. And because the new hook never schedules a render, `habits/queries.ts:7` and `app/plan/day.tsx:7` keep yesterday's date on a screen left open past midnight until something else re-renders it. The Today tab doesn't have that problem.

Why it matters

Dates are where this app's bugs have clustered (QA-308, CQ-027). Three helpers with different turn-over behaviour make the next midnight bug likely, and the name clash makes it easy to import the wrong one.

Recommendation

Rename the query hook `useTodayQuery`. Move `useLocalToday`'s timer into `use-today.ts` and build `useToday/useThisMonth` on it. Replace the ten inline calls. The existing `time-zone-use.test.ts` already guards the zone argument.

Effort

S

CQ-047 · Leftovers from this round's refactors

Low · Maintainability (owner rules) · Effort M · New

Where / what

- **The outbox queue is now the largest source file.** `apps/mobile/src/lib/outbox-queue.ts` is 534 lines (R3 335, +59%, 67% of the ceiling). It does three separate jobs: the send loop (`run`, `finish`, `cancel`), stuck groups (`park`, `parkedHeadFor`, `requeue`, `discardStuck`, `restoreStuck`, `stuckGroup`) and persistence (`persist`, the side key, `readEarlier`, `queueSave`, `durable`). It's the most data-critical module in the app, and it's growing faster than any other.
- **ToggleRow isn't adopted everywhere.** `c6a8012` extracted it, but `goal-sheet.tsx:133-141`, `debt-sheet.tsx:180-188` and `budget-edit-sheet.tsx:96-105` still use a raw `Switch` with `trackColor.true = colors.mint`. `ToggleRow` uses `ink` on purpose ("On reads like a selected

chip"). Three files, against the owner's 3-file rule, and two "on" colours.

- **Design rules are enforced by grep tests, each with its own file walker.** `colour-meaning.test.ts:7`, `type-scale.test.ts:7` and `time-zone-use.test.ts:7` each define `files()`. `time-zone-use.test.ts:37-65` hand-writes an argument counter that doesn't skip comments or regex literals. The repo already ships `typescript`, whose parser counts arguments exactly.

Recommendation

Move persistence into `outbox-store.ts` (side key, merge on load, ordered saves, `durable`) and stuck groups into `outbox-stuck.ts`, with `OutboxQueue` composing them. Do it in the same change as CQ-043, which touches the group rule anyway. Move the three toggle rows to `ToggleRow`, adding a `layout` option if the budget line needs it inline. Put `sourceFiles()` in one test helper, and count arguments with `ts.createSourceFile`.

Effort

M

CQ-041 · One stranded doc line (carried, narrowed)

Low · Maintainability / docs · Effort S · Partially fixed

Where / what

`apps/mobile/src/lib/outbox-policy.ts:86`: `/** Whether a failed attempt could have changed data on the server before it failed. */` still sits right above `STUCK_AFTER`'s doc block. Its function, `mayHaveReachedServer` (`:138`), has no doc, so editors show two stacked comments on the constant. The other five items were fixed (section 4). The lazy cycle `db/schema/time.ts:11` ↔ `money.ts:11` is still the only one in the repo, and it's harmless.

Recommendation

Move the line to `:138`.

Effort

S

5.4 Testing

CQ-042 · The new tests check behaviour with real inputs; a few paths still have none (carried, narrowed)

Low · Testing · Effort M · Partially fixed

Where

`apps/api/src/modules/ai/ask.ts` (62.68% lines; 114-187 uncovered), `packages/shared/src/quick-add.ts:151-186` (`describeDraft`; the file is at 52.69% lines), `apps/mobile/src/features/quick-add/quick-add-requests.ts` (72.90% lines), `apps/mobile/src/lib/http.ts` (61.54% funcs)

What's wrong

This round's tests are good. They assert outcomes, not calls: what's on the "disk", what was sent and in what order, the TanStack cache contents after a locked restore, and UPDATE counts. They also use the shapes production produces. Gaps remain:

- The "ask my data" tools (`spendByCategory` , `accountBalances` , `budgetStatus` and others) never run in a test. This was the fourth case round 3 asked for.
- `describeDraft` builds the one-line preview shown above Save on the capture path ("Expense ₺250 · Food · Cash"). It has no test. The split put it in a 186-line file, so the gap now shows up as 52.69% line coverage.
- The coverage table reports `server-clock.ts` at 63.89% lines, though its tests run every line. The test loads a second copy of the module (`?cold-start`), and bun seems to report that copy. Read that figure with care.

Recommendation

- Given a fake OpenAI that calls each ask tool once, when `/ai/ask` runs, then each answer uses only default-currency rows.
- Given an expense draft with a category and account, when `describeDraft` runs, then it returns "Expense ₺250 · Food · Cash". The same for a someday task, an evening task and a 1h30m time entry.
- Given a stuck `PATCH /categories/C` , when an expense in C is queued, then it is sent (CQ-043).

Effort

M

6. Strengths: keep these

- **Fixes in layers, at the root cause.** The clock bug was closed in three places: the phone ignores impossible times, a bad saved offset is refused on restore, and the server stops trusting pre-2024 stamps. Any one of them would have stopped the data loss.
- **Tests that use production's inputs.** A header-less 502 goes through the real ky instance. The cache test runs the real TanStack persister over the real encrypted storage, with a cipher that can be locked. The durability tests model the disk, a slow start and a failed save, and assert what ends up stored.
- **Undo that needs nothing from the reply.** `POST /recurring/:id/unpay` takes the pay's own `transactionId` and `dueAt`, deletes only what that pay logged, and moves the date back only if that pay moved it. It's safe to repeat and can be queued offline, and the phone cancels the pay instead if it hasn't been sent yet.
- **Narrow write paths for the assistant.** Budget drafts and their Undo write one line (`PUT /budgets/:month/:categoryId`), so a draft saved days later can't revert other lines. Settings changes go through a typed allow-list, record their previous values for Undo, and refuse the whole request if any field is off the list.
- **Durability that's explicit.** `durable()` means "read, and saved without error". The widget hands over its offline taps against that promise, and a write queued before the saved queue is readable goes to a side key instead of overwriting it.
- **Owner's rules kept.** No file is over 800 lines. There are no barrels and 0 TODOs. Every value export is used. New shared components take named variants (`IconButton shape`, `DateField kind/span`). The design rules (time zone, type scale, colour meaning) have tests that fail on a new violation.
- **Commit discipline.** All 36 commits are Conventional Commits, each one logical change. 35 carry a `Refs:` trailer, and none has co-author or agent lines. 801dd5f is a good example: a 5-line fix with its test, and a body that explains why 413 is final.
- **Unchanged strengths.** One SQLite file per user, integer money with CHECKs, zod at every boundary, typed RPC, no secrets in git (pattern scan clean), backups that check their own integrity, 0 non-null assertions, 1 justified `any`.

7. Action plan

Refactoring roadmap

When	Items
Now (this week)	CQ-043 : make writes depend on the ids a stuck group creates, and on the same record path, not on every id they mention. Describe groups by kind, and say what Discard drops. · CQ-044 : fail the audit when <code>bun audit</code> returns nothing. · CQ-045 : stop sharing the <code>http_error</code> code between the API and the phone.
Next (this month)	CQ-046 : one module and one name for "today". · CQ-037 : handle a failed token save. · CQ-047 : split <code>outbox-queue.ts</code> along its seams (with CQ-043), finish the ToggleRow migration, share the grep-test walker. · CQ-042 : ask tools and <code>describeDraft</code> tests.
Later	CQ-041 : move the stranded doc line. Keep an eye on <code>outbox-queue.ts</code> and <code>recurrence.ts</code> (484) relative to the ceiling.

Prioritised table

Order	ID	Title	Severity	Effort
1	CQ-043	A stuck edit holds back later writes that only name its record; Discard drops them	Medium	M
2	CQ-044	The CI audit passes when <code>bun audit</code> can't run	Low	S
3	CQ-045	API and phone both use the <code>http_error</code> code; such a write retries forever	Low	S
4	CQ-037	A failed refreshed-token save is swallowed	Low	S
5	CQ-046	Three "today" helpers; two hooks named <code>useToday</code>	Low	S
6	CQ-047	<code>outbox-queue.ts</code> 534 lines with three jobs; ToggleRow not adopted; grep-test walkers	Low	M
7	CQ-042	ask tools and <code>describeDraft</code> untested	Low	M
8	CQ-041	Stranded doc line in <code>outbox-policy.ts</code>	Low	S

8. Appendix

A. Raw command output (trimmed)

```
$ bunx biome check .
Checked 555 files in 374ms. No fixes applied.          exit 0
$ bun run typecheck
@tick-taka/shared typecheck: Exited with code 0
@tick-taka/api typecheck: Exited with code 0
@tick-taka/mobile typecheck: Exited with code 0
$ bun run test
@tick-taka/shared test: 247 pass, 0 fail, 519 expect() (13 files)
@tick-taka/mobile test: 291 pass, 0 fail, 865 expect() (56 files)
@tick-taka/api test:    236 pass, 0 fail, 848 expect() (31 files) [9.65s]
$ bun test --coverage (All files: % Funcs | % Lines)
api      88.20 | 94.50  lowest: ai/ask.ts 62.68, categories/routes.ts 79.25, schema/time.ts 79.72
shared   84.16 | 95.08  quick-add.ts 52.69, tiki.ts 69.09, quick-add-task.ts 98.43
mobile   90.97 | 93.05  server-clock.ts 63.89*, quick-add-requests.ts 72.90, plan-
notifications.ts 80.82
$ bun run audit
accepted GHSA-vfj7-8cjl-p6xm braces ... accepted GHSA-vcc3-ghjq-m6fr decode-uri-component ...
accepted GHSA-67mh-4wv8-2f99 esbuild ... accepted GHSA-86w9-cpqp-85rv node-forge ...
accepted GHSA-rj75-hqrm-r3gf postcss-selector-parser ...
No new advisories (5 accepted).                      exit 0
$ bunx expo export --platform android
Android Bundled 51854ms apps/mobile/index.ts (2855 modules) · index-....hbc (7.7MB)  exit 0
```

* See CQ-042: the figure probably reflects the second module copy the test loads.

B. Code health counts (source only: apps/api/src, apps/mobile/src, packages/shared/src)

Pattern	R1	R2	R3	R4	Notes / locations
any	1	1	1	1	api/src/lib/crud.ts:46 , with a justified biome-ignore
@ts-ignore / @ts-expect-error	0	0	0	0	
biome-ignore	4	7	7	7	crud.ts:45; message-bubble.tsx:24,65,211; chat-composer.tsx:66; use-notification-scheduler.ts:107; habit-list.tsx:116. All give a reason.
as X casts (AST, excl. as const)	~187*	135	144	143	*R1 was counted by regex. Top in source: resource-routes.ts 10, crud.ts 9, assistant/tools.ts 9, outbox-policy.ts 7, assistant/actions.ts 6. as const : 96. Tests: 100.
as unknown as	1	1	1	1	mobile/src/lib/receipts.ts:47 (RN FormData file)
Non-null !	44	60	0	0	The rule is an error outside tests (tests: 107)
TODO / FIXME / HACK	0	0	0	0	Source and tests

Pattern	R1	R2	R3	R4	Notes / locations
<code>console.*</code>	5	5	6	11	<code>api log.ts:7,8</code> (the sink). Mobile <code>console.warn</code> at <code>secure-storage.ts:54,59,72</code> ; <code>user-data.ts:24</code> ; <code>sync-pull.ts:68</code> ; <code>outbox.ts:46</code> ; <code>use-export-data.ts:31</code> ; <code>use-notification-scheduler.ts:89</code> ; <code>use-widget-sync.tsx:67</code> . Each logs a failure that was swallowed before. The phone has no log module of its own.
<code>.catch(() => {})</code> / empty <code>catch {}</code>	20 / 2	27 / 2	24 / 2	23 / 2	Best-effort calls (haptics, sounds, notifications, widget redraw). The outbox's swallow is gone. The 2 truly empty catches are <code>agent.ts:85,89</code> (a display-only JSON parse); 4 more have a comment.
Runtime import cycles	0	1	1	1	<code>db/schema/time.ts ↔ money.ts</code> (lazy FK thunks, safe). The quick-add siblings import back types only.
Files over 800 lines	0	0	0	0	Largest source file: <code>mobile/lib/outbox-queue.ts</code> , 534. Largest test: <code>api/test/money.test.ts</code> , 548.
Barrel index files	0	0	0	0	
Duplicate 8-line blocks	n/a	10	3	2	Each spans 2 files: the emoji picker (<code>category-sheet.tsx:121</code> , <code>area-sheet.tsx:109</code>) and the area row (<code>areas-editor.tsx:95</code> , <code>plan-home.tsx:149</code>). None spans 3 or more.
Exports referenced nowhere	n/a	2 / 10	1 / 14	0 / 13	Values / types. The types are store aliases (JobRunStore, SessionStore, UsageTotalStore, SessionManager) and schema-inferred aliases (TaskStatus, AreaCreate, ErrorBody and others).
Committed secrets / data files	0	0	0	0	Only the two <code>.env.example</code> files are tracked. The local APK is git-ignored. No key patterns found.

C. Largest functions and files (TypeScript AST)

Lines	Function	R3	Lines	Largest source files (vs 800)
297	TaskSheet (<code>tasks/task-sheet.tsx:34</code>)	297	534	<code>mobile/src/lib/outbox-queue.ts</code> (R3 335)
282	TransactionSheet (<code>money/transaction-sheet.tsx:35</code>)	285	484	<code>shared/src/recurrence.ts</code>
263	taskService (<code>api tasks/service.ts:38</code>)	263	385	<code>mobile/.../widget/safe-to-spend-widget.tsx</code> (R3 336)
254	InsightsScreen (<code>review/insights-screen.tsx:44</code>)	261	360	<code>api/.../ai/assistant/tools.ts</code>
227	SafeToSpendWidget (<code>widget/safe-to-spend-widget.tsx:159</code>)	209	343	<code>api/.../budgets/service.ts</code> (+61)
224	RecurringForm (<code>money/recurring-sheet.tsx:54</code>)	231	333	<code>shared/src/schemas/money.ts</code>
216	useAssistant (<code>assistant/use-assistant.ts:66</code>)	216	330	<code>mobile/.../tasks/task-sheet.tsx</code>
			316	<code>mobile/.../money/transaction-sheet.tsx</code>
			303	<code>shared/src/defaults.ts</code>

Lines	Function	R3
210	createToolRunner (assistant/tools.ts:151)	210
210	WeeklyReview (review/weekly-review.tsx:26)	210
207	PlanHome (plan/plan-home.tsx:24)	207

Lines	Largest source files (vs 800)
300	api/.../tasks/service.ts

All are under the ceiling. SQL: 0004_constraints.sql 428 and 0000_init.sql 332, both unchanged. quick-add.ts went from 649 to 186.

D. Requirement checklist (engineering commitments from the docs)

Commitment (source)	Status	Evidence
TypeScript strict everywhere (PLAN, owner rules)	Met	<code>tsconfig.base.json</code> : strict, noUncheckedIndexedAccess. <code>scripts/</code> isn't typechecked (CQ-044).
800-line ceiling; no barrels (owner rules)	Met	Appendix B and C. Watch outbox-queue.ts (CQ-047).
Markup in 3+ files is a shared component with named variants (owner rules)	Mostly met	MonthStepper, IconButton, ToggleRow, DateField, useChartAxis. Three raw Switch rows remain (CQ-047).
Conventional Commits, no attribution, one change per commit (owner rules)	Met	36 of 36
Money as integers (PLAN)	Met	<code>*_minor</code> integer columns with CHECKs
Per-user data isolation (README)	Met	<code>user-registry.ts</code> , <code>user-scope.ts</code>
Offline queue never loses writes (PLAN Phase 7, README)	Partial	The round-3 losses are fixed and tested. CQ-043 can stall writes, and Discard can drop them.
Data encrypted at rest on the phone (README)	Met	The cache is never written in plain text, and only the outbox may fall back to plain text. The export file is removed (timing Unverified).
Login rate limit, 5 per 15 min per IP (PLAN)	Met in code	Production <code>TRUST_PROXY</code> Unverified
Nightly backups, 14 kept, off-site copy (PLAN, README)	Met in code	<code>BACKUP_OFFSITE_CMD</code> on the server Unverified
No AI feature saves records without confirmation (spec)	Partial (documented)	Money changes are drafts, and deletions wait for a tap. Settings are now limited to an allow-list and come with Undo. Task, habit and timer changes still apply at once, as a documented product decision.
Dependency advisories watched (README)	Partial	The allow-list is in CI, but the check fails open (CQ-044).
Every phase ends with tests passing (PLAN)	Met	<code>check.yml</code> on push to main and on PRs

E. File inventory

Workspace	Source files	Source lines	Test files	Test lines
apps/api	105	10,111	35 (31 test + 4 helpers)	5,489
apps/mobile	299	22,778	56	4,078
packages/shared	21	3,319	13	1,486
Total	425	36,208	104	11,053

Round 3 had 411 source files with 34,944 lines and 91 test files with 9,314 lines. Migrations are still 0000 to 0005. The new unpay and per-line budget routes needed no schema change.

F. Previous-findings tally

Status	IDs
Fixed (7)	CQ-012, 020, 022, 036, 038, 039, 040
Partially fixed (3)	CQ-037, 041, 042 (carried, narrowed)
Not fixed / Regressed	none

G. Probe scripts (run from `_cq_tmp/`, then deleted)

```
// probe.test.ts (run with bun test from apps/mobile; imports the real modules)
// A: CQ-045
const error = apiErrorFrom(400, { error: { code: "http_error", message: "Malformed JSON in
request body" } });
describeFailure(error);           // { unreachable: true } → classifyFailure: "retry",
isServerFault: false
// B: CQ-043. A real OutboxQueue; send() answers 500 for /categories/* and 200 otherwise
await queue.load();
queue.enqueue({ method: "PATCH", path: `/categories/${cat}`, body: { name: "Food" } });
queue.enqueue({ method: "POST", path: "/transactions", body: { id: newId(), categoryId: cat,
amountMinor: 2000 } });
// ...after 8 faults the PATCH is parked; then a second expense in the same category is queued
queue.stuck();                    // [PATCH /categories/C, POST /transactions, POST
/transactions]; 0 transactions sent
// scan.ts / graph.ts: TypeScript AST counts, Tarjan SCC over runtime imports (relative, @/ and
// @tick-taka/shared), 8-line windows of normalised lines ≥ 200 chars, exports grepped across
apps/* and packages/.
```